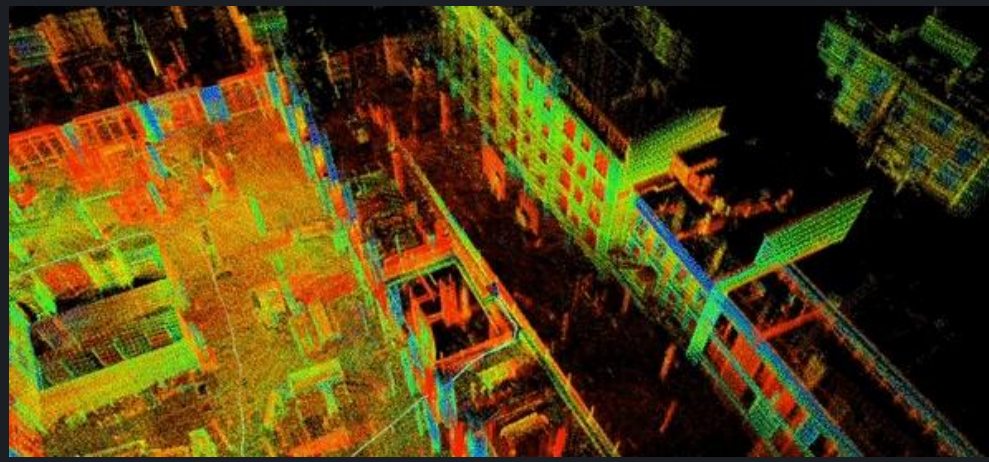
Robust 3D Change Detection for Construction Progress Monitoring

Vishwanath Venugopal, CE23B124
Shaik Koduru Asif, CE23B114

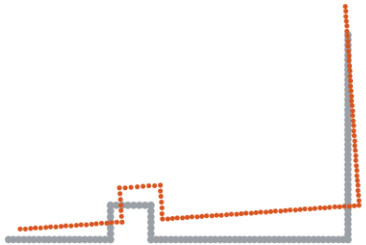
Dataset: **MultiChange3D + ConSLAM**



Ground-truth laser scan of the ConSLAM site

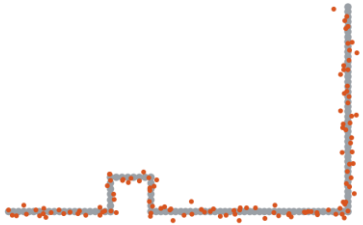
Why comparing two scans is hard

Same wall, scanned twice. Grey is scan 1, orange is scan 2.



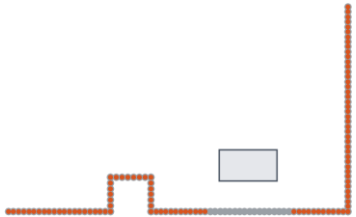
Registration error

The scans are not perfectly aligned, so a wall that did not move looks shifted.



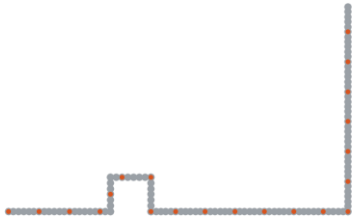
Sensor noise

Points scatter around the true surface.



Occlusion

Something blocks the view, so part of the wall is missing in one scan.



Different density

One scan has far fewer points than the other.

A plain scan-to-scan comparison marks all of this as change. The real construction changes get buried in false alarms.

What we want to do

Objective

Build a 3D change-detection pipeline that finds added, removed, or unchanged regions between two scans between two LiDAR scans of the same site, and check how well it works when the scans are imperfect.

What is new

We are not proposing another model. The published benchmark studies how methods generalise across scenes. We study how they hold up as registration error, sparsity, noise and occlusion increase.

Two scans

same area, different dates

→

Use co-registered pairs

pairs ship co-registered

→

Detect change

C2C, M3C2, Siamese PointNet

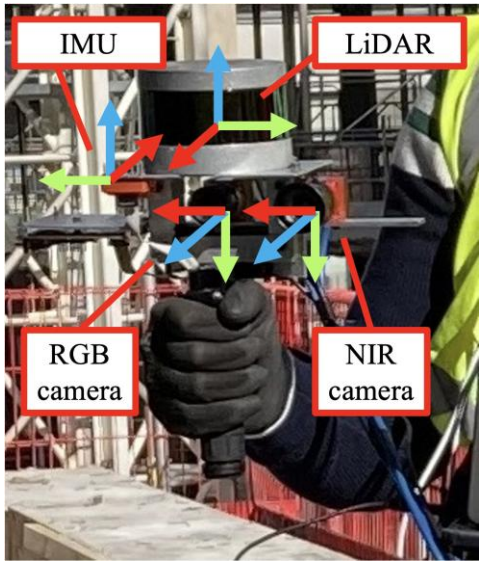
→

Label points

added, **removed**, **unchanged**

The datasets

- MultiChange3D (ETH Zurich and FBK): a benchmark for 3D geometric change detection, published at ISPRS 2026
- We use its Bike parking construction scene: MLS, 4 epochs, 6 epoch pairs, about 5 million points
- Every point is labelled unchanged, added or removed
- ConSLAM: real construction site scanned monthly by a Cambridge team, five sequences, TLS ground truth
- **MultiChange3D → quantitative evaluation**
  ConSLAM → qualitative real-site validation



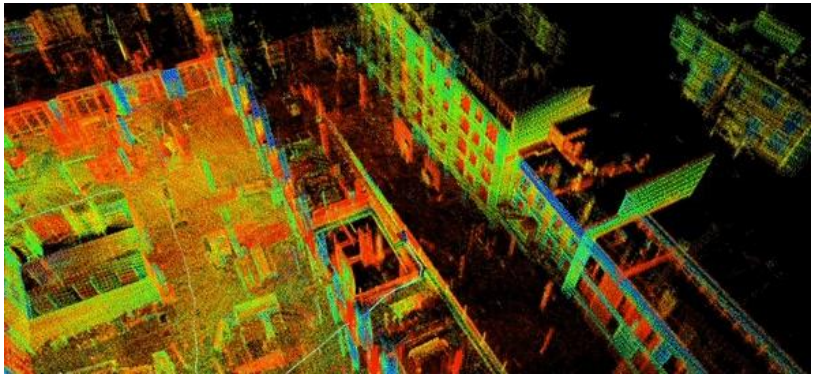
ConSLAM hand-held scanner



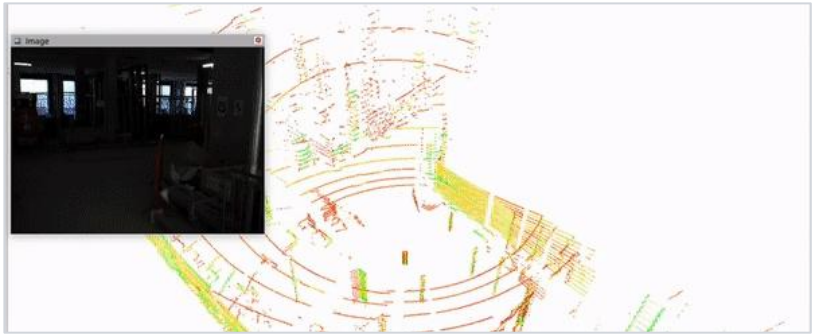
ConSLAM terrestrial laser scanner

Photos: ConSLAM project page (Trzeciak et al.)

What each dataset is for



Ground-truth TLS scan: dense and clean



One hand-held LiDAR sweep: sparse

MultiChange3D

Co-registered pairs with per-point change labels, evaluation code, and published baselines to check our numbers against.

ConSLAM

A real site over several months, but built for SLAM, so it carries no point-level change labels.

How we split them

Every number comes from MultiChange3D. ConSLAM is used only for qualitative change maps on a real site.

Frames from the ConSLAM project video (Trzeciak et al.)

How we detect change

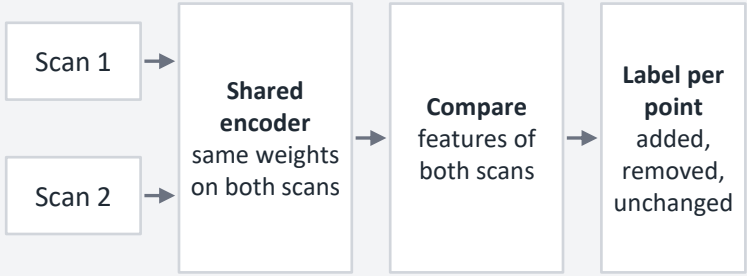
Baseline: C2C and M3C2



Dashed lines: distance to the nearest point in scan 1

- Distance from each point to the nearest point in the other scan
- Far from scan 1 means added, far from scan 2 means removed
- Threshold, then DBSCAN. M3C2 uses normals.

Learning-based: Siamese network



- The same encoder runs on both scans
- Encoder: lightweight PointNet-style, plain PyTorch
- Split by location so test areas are unseen

Both start from the same registered scan pairs and are scored the same way.

Testing how well they hold up

Imperfection	How we simulate it	Planned experimental levels
Registration error	Random rotation and shift applied to scan 2	0 to 5 degrees, 0 to 50 cm
Sparsity	Randomly keep a fraction of the points	100%, 50%, 25%, 10%
Noise	Add Gaussian noise to every point	Increasing sigma
Occlusion	Delete points in a wedge or slab	Increasing size

One factor at a time, then combined

Precision | Recall | F1 | IoU

Main result: each metric plotted against the level, one curve per method.

Sparsity example (schematic)



Implementation plan

1 Prepare data

Download the bike parking scene, load the point-cloud pairs and labels

2 Verify

Reproduce a published C2C score on the same scene

3 Baseline

C2C nearest-neighbour distance with tuned thresholds, then M3C2

4 Degradations

Code for registration error, sparsity, noise and occlusion

5 First curves

Run C2C and M3C2 at every level: P, R, F1 and IoU

6 Train model

Siamese PointNet, with a simpler fallback

7 Comparison

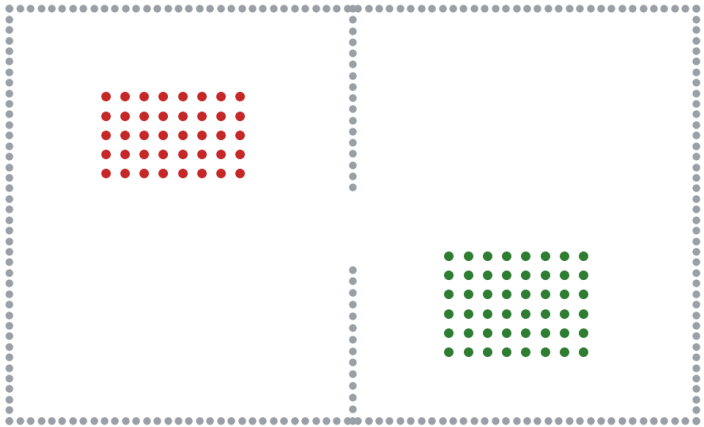
Learning model through the same degradation harness

8 Results

ConSLAM maps, plots, 3D visualisation, write-up

Steps 1 to 5 already give a complete working study with two methods. Tools: Python, Open3D, NumPy, PyTorch, CloudCompare.

Expected outcome and future scope



added removed unchanged

Planned 3D output, shown here as a top-view sketch

Expected outcome

A system that finds and shows real construction change between scans, and clear evidence of which approach is more reliable on imperfect data.

Beyond this project (Future Scope)

- Say what changed: wall, slab, column, scaffolding
- Compare scans against the BIM design model
- Turn changes into progress figures, such as percent complete
- Track many scans over time, not only pairs
- Train with the degradations as augmentation
- Extend to the other MultiChange3D scenes and sensors